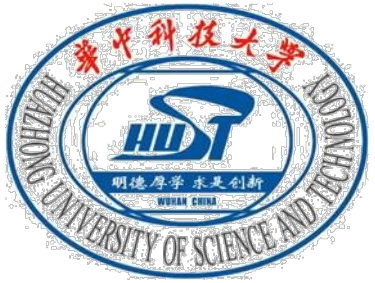
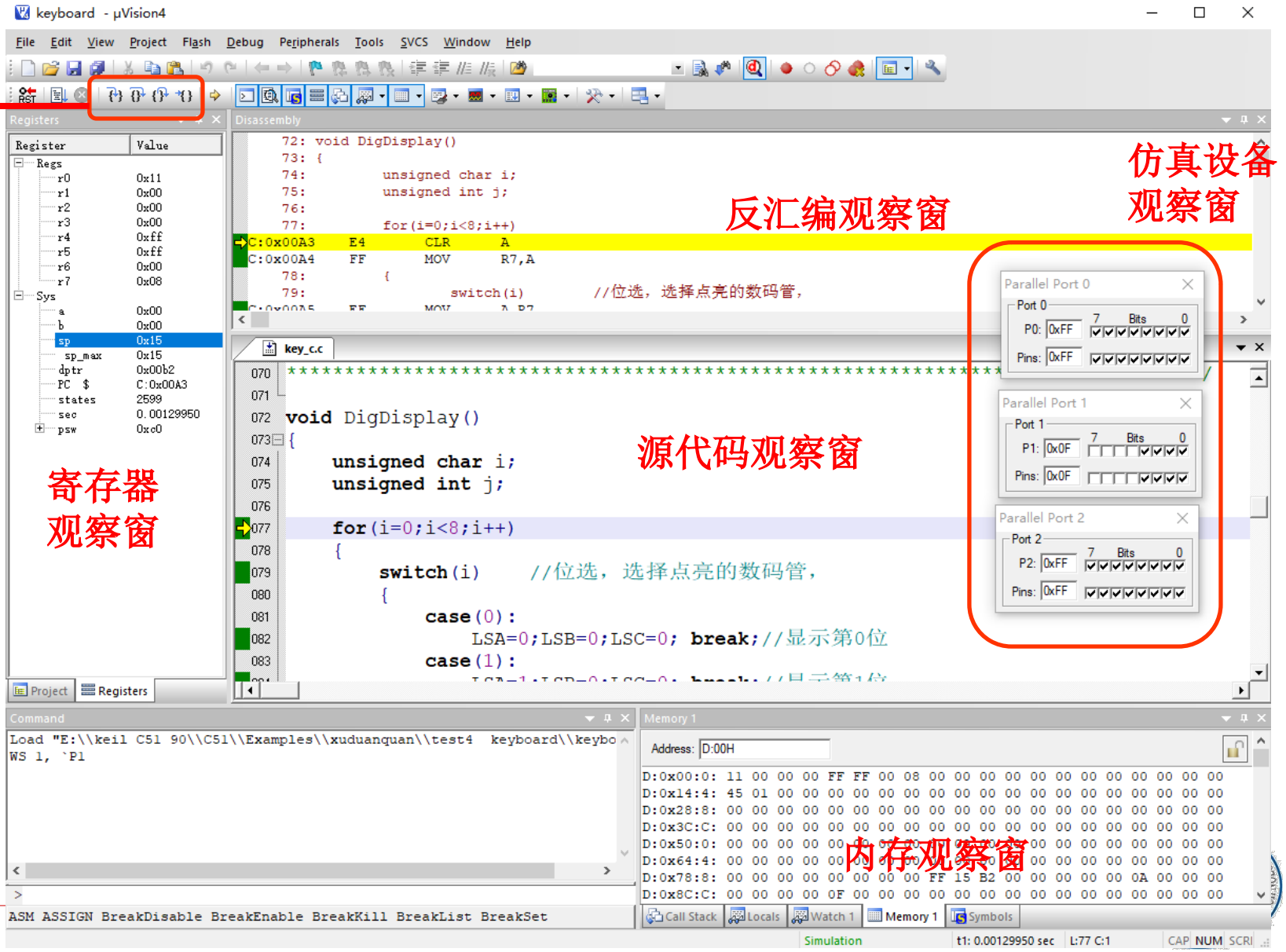
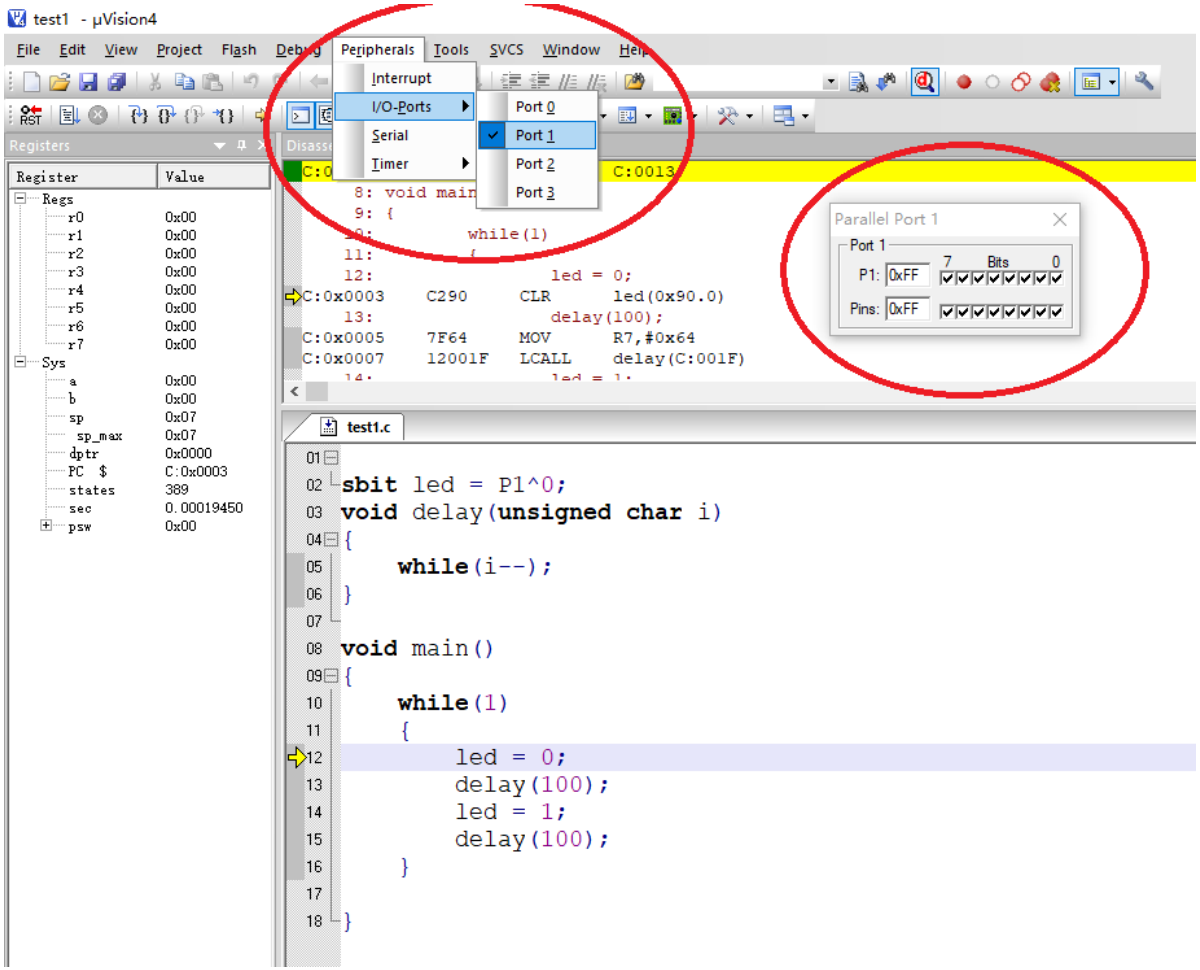
51 单片机键盘与数码管试验



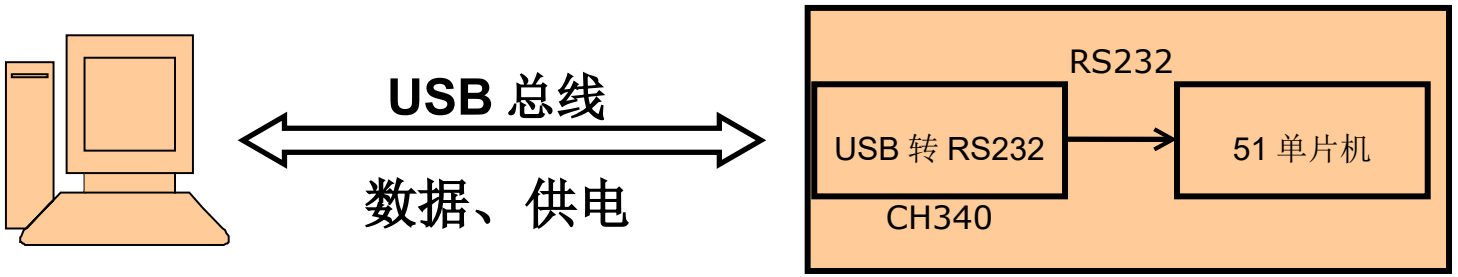
Keil 使用回顾


keyboard - µVision4
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
调试控制：
单步
跨越单步
跳出函数
执行到光标处
Registers
Register Value
Regs
r0 0x11
r1 0x00
r2 0x00
r3 0x00
r4 0xff
r5 0xff
r6 0x00
r7 0x08
Sys
a 0x00
b 0x00
sp 0x15
sp_max 0x15
dptr 0x00b2
PC $ C:0x00A3
states 2599
sec 0.00129950
psw 0xc0
寄存器观察窗
Disassembly
72: void DigDisplay()
73: {
74: unsigned char i;
75: unsigned int j;
76:
77: for(i=0;i<8;i++)
C:0x00A3 E4 CLR A
C:0x00A4 FF MOV R7,A
78: {
79: switch(i) //位选，选择点亮的数码管，
反汇编观察窗
仿真设备观察窗
Parallel Port 0
Port 0
P0: 0xFF
Pins: 0xFF
Parallel Port 1
Port 1
P1: 0x0F
Pins: 0x0F
Parallel Port 2
Port 2
P2: 0xFF
Pins: 0xFF
key_c.c
070 ********************
071
072 void DigDisplay()
073 {
074 unsigned char i;
075 unsigned int j;
076
077 for(i=0;i<8;i++)
078 {
079 switch(i) //位选，选择点亮的数码管，
080 {
081 case(0):
082 LSA=0;LSB=0;LSC=0; break;//显示第0位
083 case(1):
源代码观察窗
Project Registers
Command
Load "E:\\keil C51 90\\C51\\Examples\\xuduanquan\\test4 keyboard\\keybo
WS 1, `P1
ASM ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet
Memory 1
Address: D:00H
D:0x00:0: 11 00 00 00 FF FF 00 08 00 00 00 00 00 00 00 00 00 00 00 00
D:0x14:4: 45 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D:0x28:8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D:0x3C:C: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D:0x50:0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D:0x64:4: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D:0x78:8: 00 00 00 00 00 00 00 00 FF 15 B2 00 00 00 00 00 0A 00 00 00
D:0x8C:C: 00 00 00 00 0F 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
内存观察窗
Call Stack Locals Watch 1 Memory 1 Symbols
Simulation t1: 0.00129950 sec L:77 C:1
CAP NUM SCRI

Keil 使用回顾


test1 - µVision4
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Interrupt
I/O-Ports
Serial
Timer
Port 0
Port 1
Port 2
Port 3
Registers
Register Value
Regs
r0 0x00
r1 0x00
r2 0x00
r3 0x00
r4 0x00
r5 0x00
r6 0x00
r7 0x00
Sys
a 0x00
b 0x00
sp 0x07
sp_max 0x07
dptr 0x0000
PC $ C:0x0003
states 389
sec 0.00019450
psw 0x00
8: void main
9: {
10: while(1)
11: {
12: led = 0;
C:0x0003 C290 CLR led(0x90.0)
13: delay(100);
C:0x0005 7F64 MOV R7,#0x64
C:0x0007 12001F LCALL delay(C:001F)
Parallel Port 1
Port 1
P1: 0xFF 7 Bits 0
Pins: 0xFF
test1.c
01
02 sbit led = P1^0;
03 void delay(unsigned char i)
04 {
05 while(i--);
06 }
07
08 void main()
09 {
10 while(1)
11 {
12 led = 0;
13 delay(100);
14 led = 1;
15 delay(100);
16 }
17
18 }

51 单片机交叉开发环境回顾

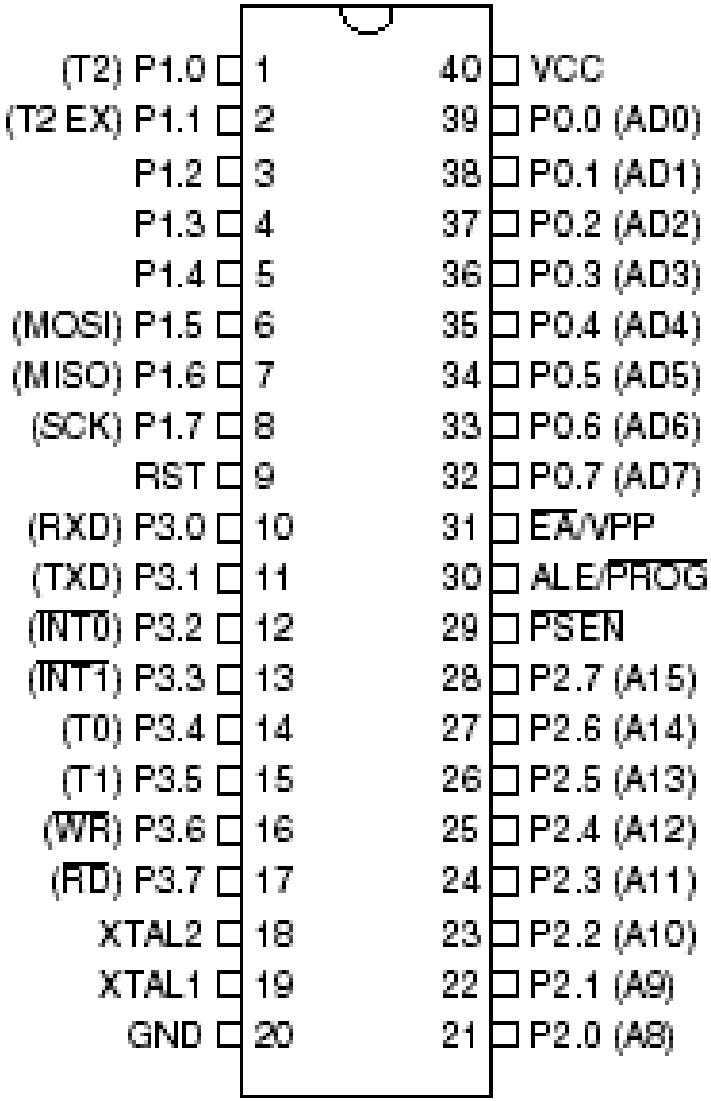


软件开发计算机
PC 机

51 单片机
开发板

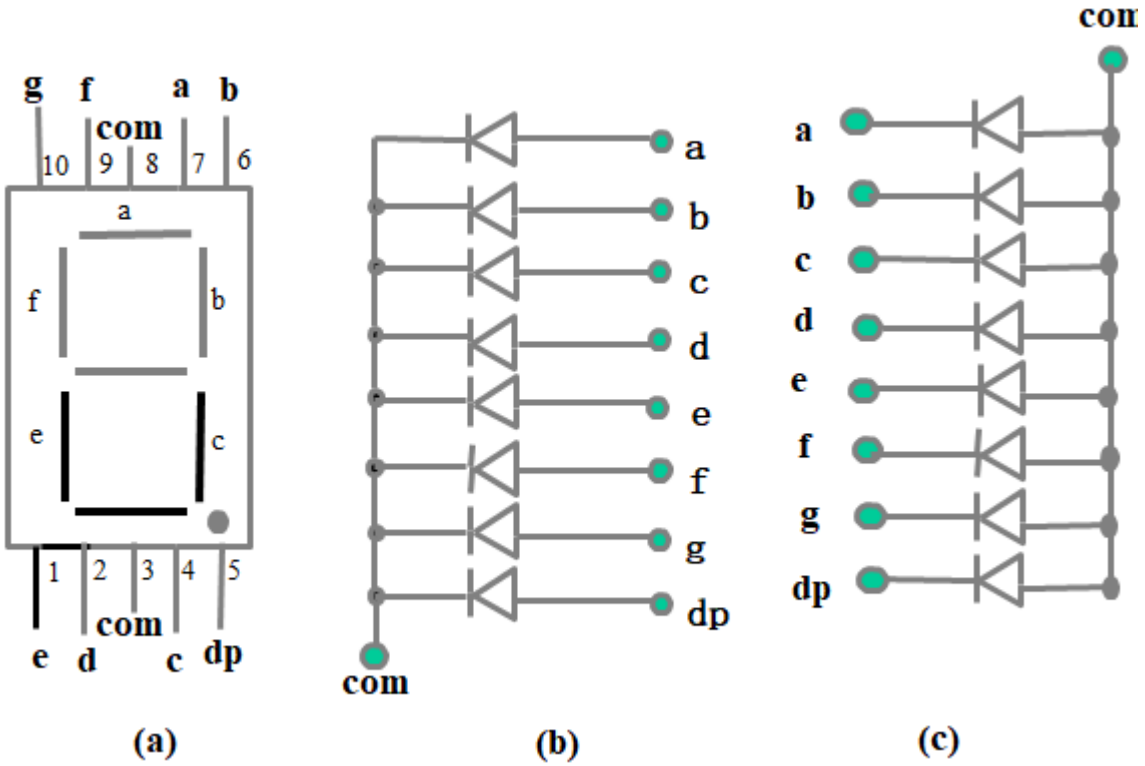
MCS-51 单片机 ---- 引脚功能

- **P0.0 ～ P0.7：** **P0** 口，数据 / 低八位地址复用总线端口。
- **P1.0 ～ P1.7：** **P1** 口，静态通用端口。
- **P2.0 ～ P2.7：** **P2** 口，高八位地址总线端口。
- **P3.0 ～ P3.7：** **P3** 口，双功能静态端口



并行接口——数码管驱动

- 七段数码管由七个条形发光二极管和一个小圆点发光二极管组成
- **COM** 为公共点，根据内部发光二极管的接线 形式，可分成共阴极型图 **(b)** 和共阳极型图 **(c)**

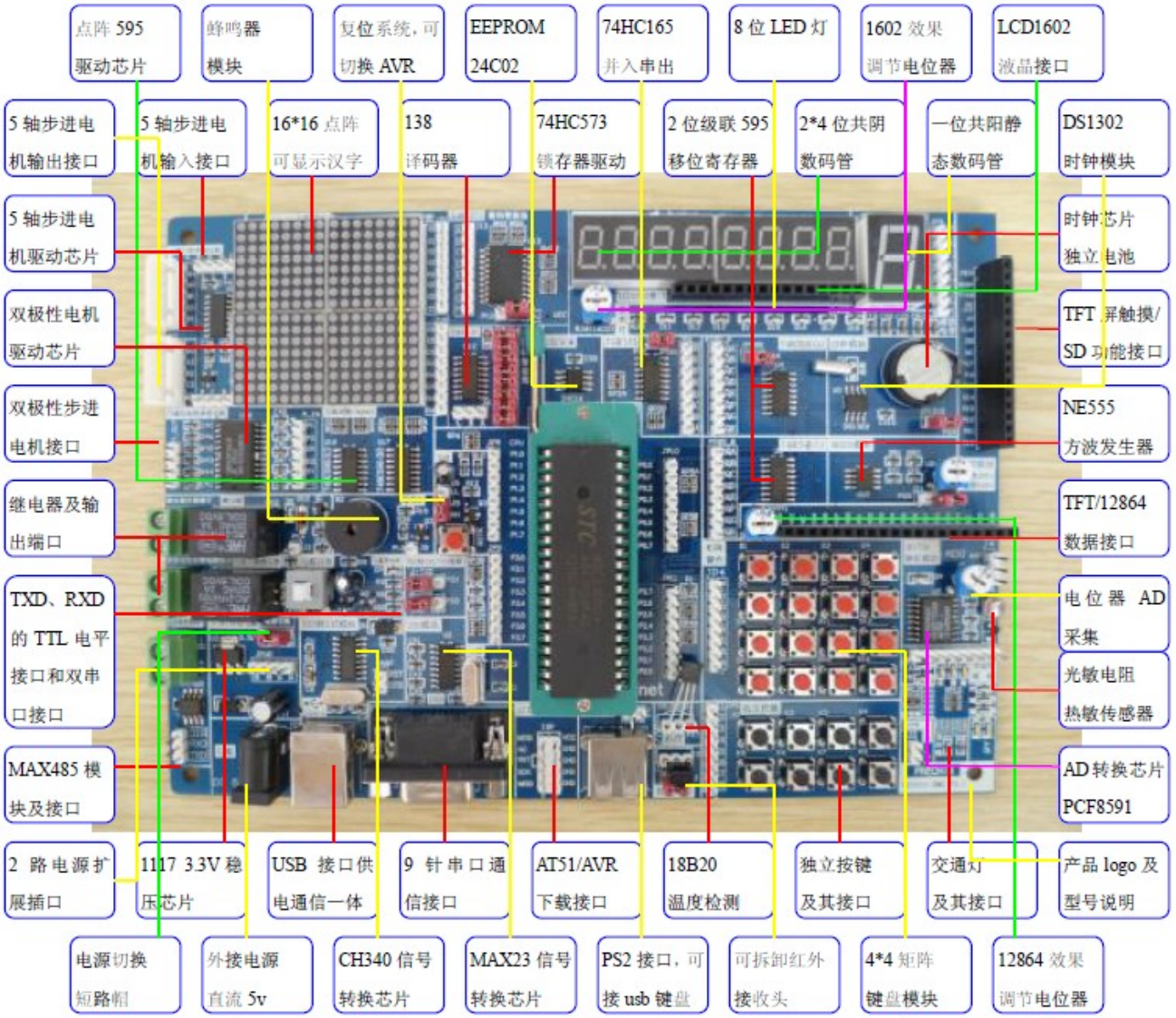


并行接口——数码管驱动

- 共阳、共阴七段数码管控制字（与接线方式相关）

显示字符	段符号 P0.7 dp	P0.6 g	P0.5 f	P0.4 e	P0.3 d	P0.2 c	P0.1 b	P0.0 a	十六进制代码 共阴	共阳
0	0	0	1	1	1	1	1	1	3FH	C0
1	0	0	0	0	0	1	1	0	06H	F9
2	0	1	0	1	1	0	1	1	5BH	A4
3	0	1	0	0	1	1	1	1	4FH	B0
4	0	1	1	0	0	1	1	0	66H	99
5	0	1	1	0	1	1	0	1	6DH	92
6	0	1	1	1	1	1	0	1	7DH	82
7	0	0	0	0	0	1	1	1	07H	F8
8	0	1	1	1	1	1	1	1	7FH	80
9	0	1	1	0	1	1	1	1	6FH	90
A	0	1	1	1	0	1	1	1	77H	88
B	0	1	1	1	1	1	0	0	7CH	83
C	0	0	1	1	1	0	0	1	39H	C6
D	0	1	0	1	1	1	1	0	5EH	A1
E	0	1	1	1	1	0	0	1	79H	86
F	0	1	1	1	0	0	0	1	71H	84
H	0	1	1	1	0	0	0	1	76H	FF ？？
P	1	1	1	1	0	0	1	1	F3H	BF

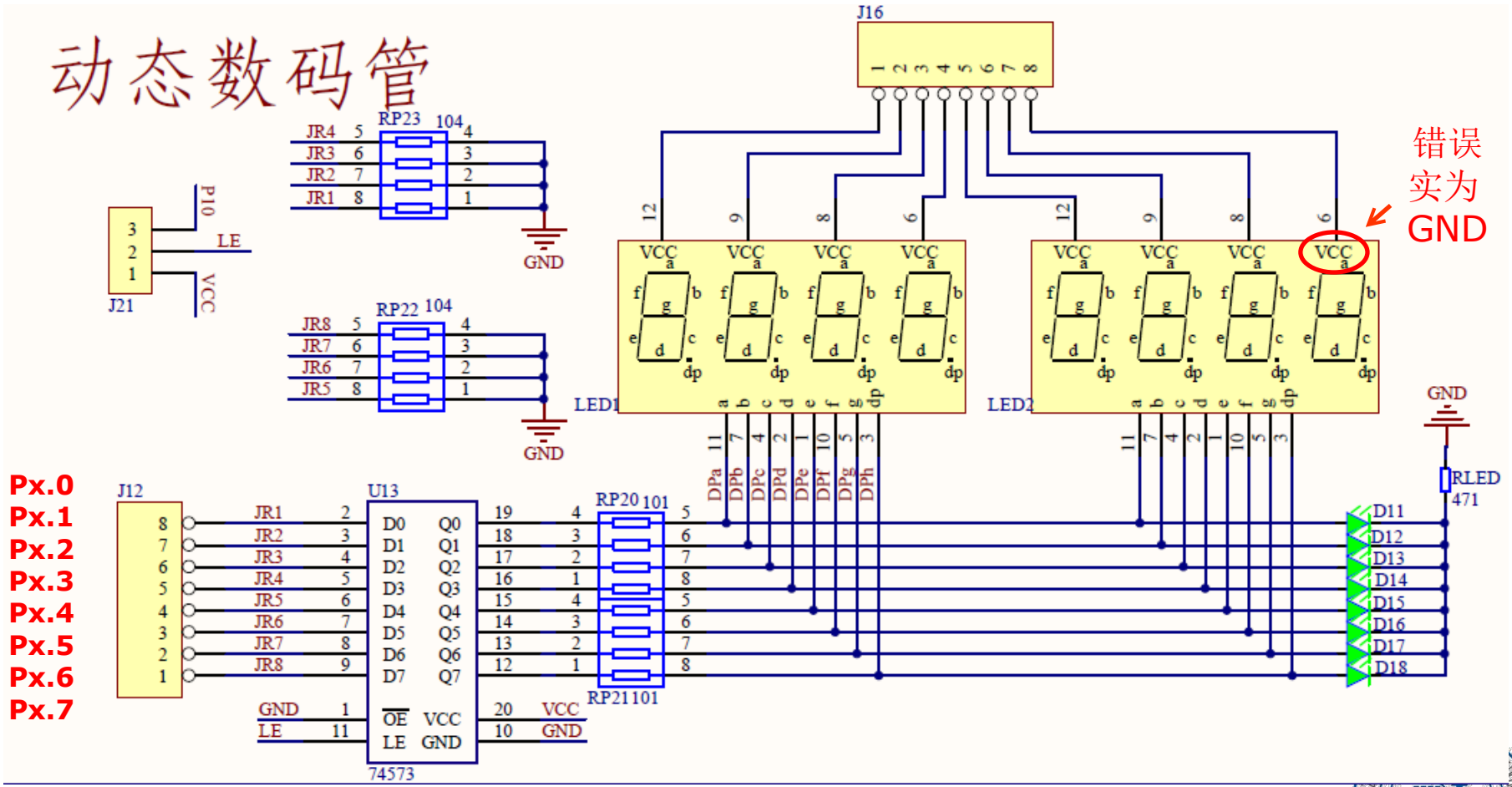
并行接口——数码管驱动


点阵 595 驱动芯片
蜂鸣器模块
复位系统，可切换 AVR
EEPROM 24C02
74HC165 并入串出
8 位 LED 灯
1602 效果调节电位器
LCD1602 液晶接口
5 轴步进电机输出接口
5 轴步进电机输入接口
16*16 点阵可显示汉字
138 译码器
74HC573 锁存器驱动
2 位级联 595 移位寄存器
2*4 位共阴数码管
一位共阳静态数码管
DS1302 时钟模块
5 轴步进电机驱动芯片
时钟芯片独立电池
双极性电机驱动芯片
TFT 屏触摸/SD 功能接口
双极性步进电机接口
NE555 方波发生器
继电器及输出端口
TFT/12864 数据接口
TXD、RXD 的 TTL 电平接口和双串口接口
电位器 AD 采集
光敏电阻热敏传感器
MAX485 模块及接口
AD 转换芯片 PCF8591
2 路电源扩展插口
1117 3.3V 稳压芯片
USB 接口供电通信一体
9 针串口通信接口
AT51/AVR 下载接口
18B20 温度检测
独立按键及其接口
交通灯及其接口
产品 logo 及型号说明
电源切换短路帽
外接电源直流 5v
CH340 信号转换芯片
MAX23 信号转换芯片
PS2 接口，可接 usb 键盘
可拆卸红外接收头
4*4 矩阵键盘模块
12864 效果调节电位器

并行接口——数码管驱动

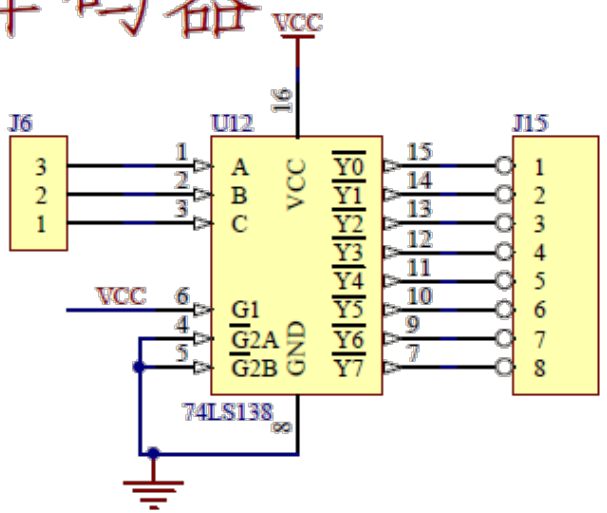
电路图见 QQ 群文件：
EM3_V30 原理图 .pdf

Px 端口
三八译码器



并行接口——数码管驱动（3-8 译码器）

138译码器

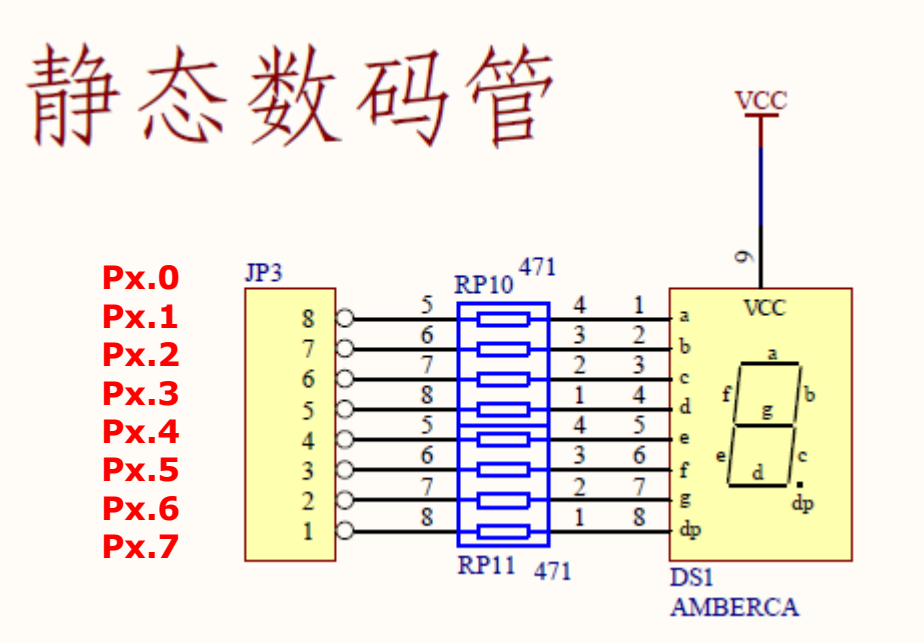


Px 端口低三位： 0->A； 1->B； 2->C 时，
对应数码管片选控制码为：

DB 0H,01H,02H,03H,04H,05H,06H,07H

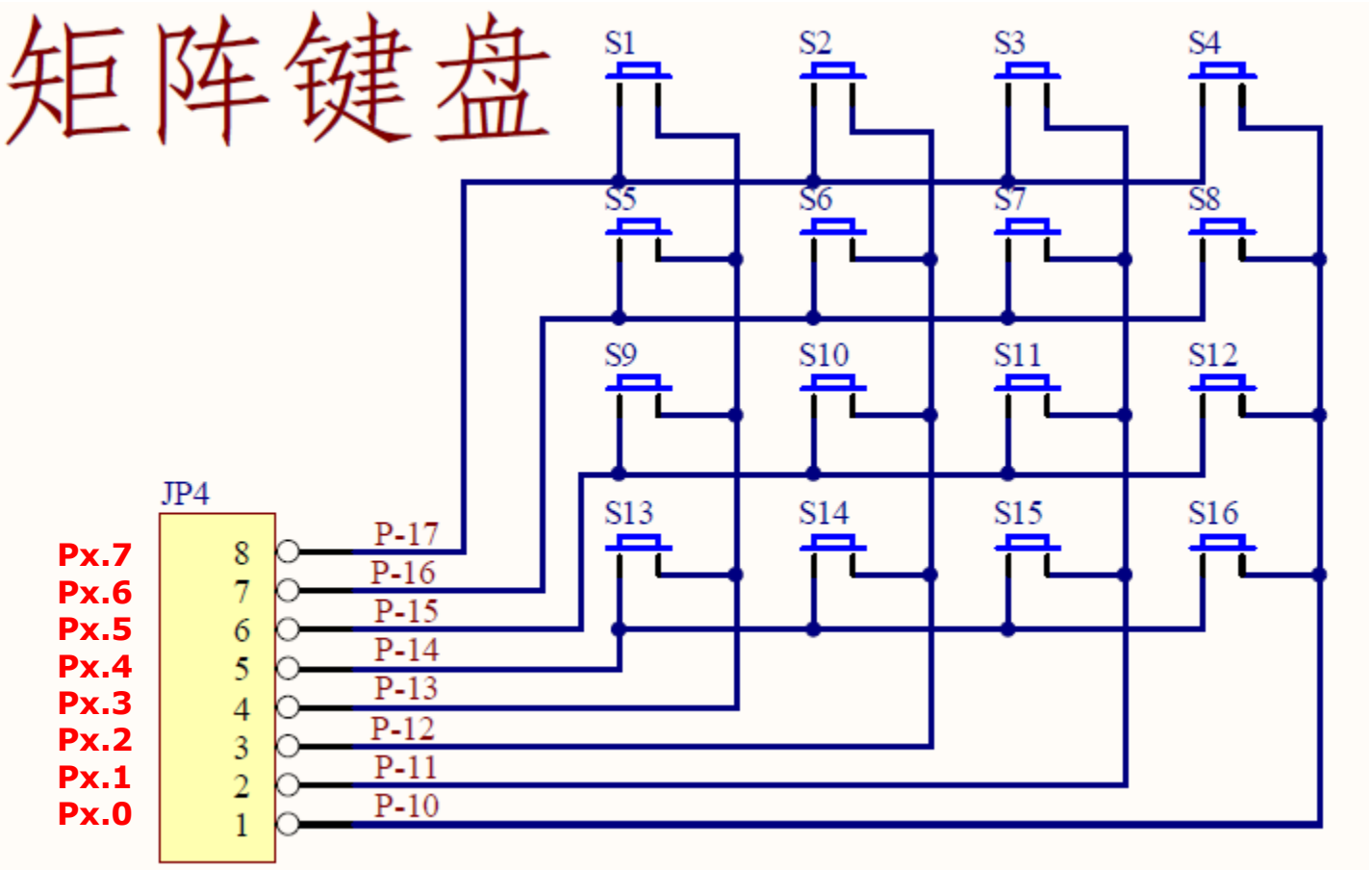
输入						输出							
G_1	G_{2A}	G_{2B}	C	B	A	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
×	H	×	×	×	×	H	H	H	H	H	H	H	H
×	×	H	×	×	×	H	H	H	H	H	H	H	H
L	×	×	×	×	×	H	H	H	H	H	H	H	H
H	L	L	L	L	L	L	H	H	H	H	H	H	H
H	L	L	L	L	H	H	L	H	H	H	H	H	H
H	L	L	L	H	L	H	H	L	H	H	H	H	H
H	L	L	L	H	H	H	H	H	L	H	H	H	H
H	L	L	H	L	L	H	H	H	H	L	H	H	H
H	L	L	H	L	H	H	H	H	H	H	L	H	H
H	L	L	H	H	L	H	H	H	H	H	H	L	H
H	L	L	H	H	H	H	H	H	H	H	H	H	L

并行接口——数码管驱动



Px.x 为低电平时对应段点亮，与共阴数码管相反

并行接口——矩阵键盘



P 端口写入 00001111(0FH)，再读 P 端口，如果为 0FH，没有按键；若有按键，对应行线为 0 。

P 端口写入 11110000(F0H)，再读 P 端口，如果为 F0H，没有按键；若有按键，对应列线为 0 。

并行接口——矩阵键盘

- **#define GPIO_KEY P1**

即用 **P1** 端口扫描键盘

- 每一行对应键码起始值为 **0**， **4**，**8**， **C**（**12**）
- 确定列后，在行起始值基础上，加 **0**， **1**， **2**， **3**

C	D	E	F
8	9	A	B
4	5	6	7
0	1	2	3

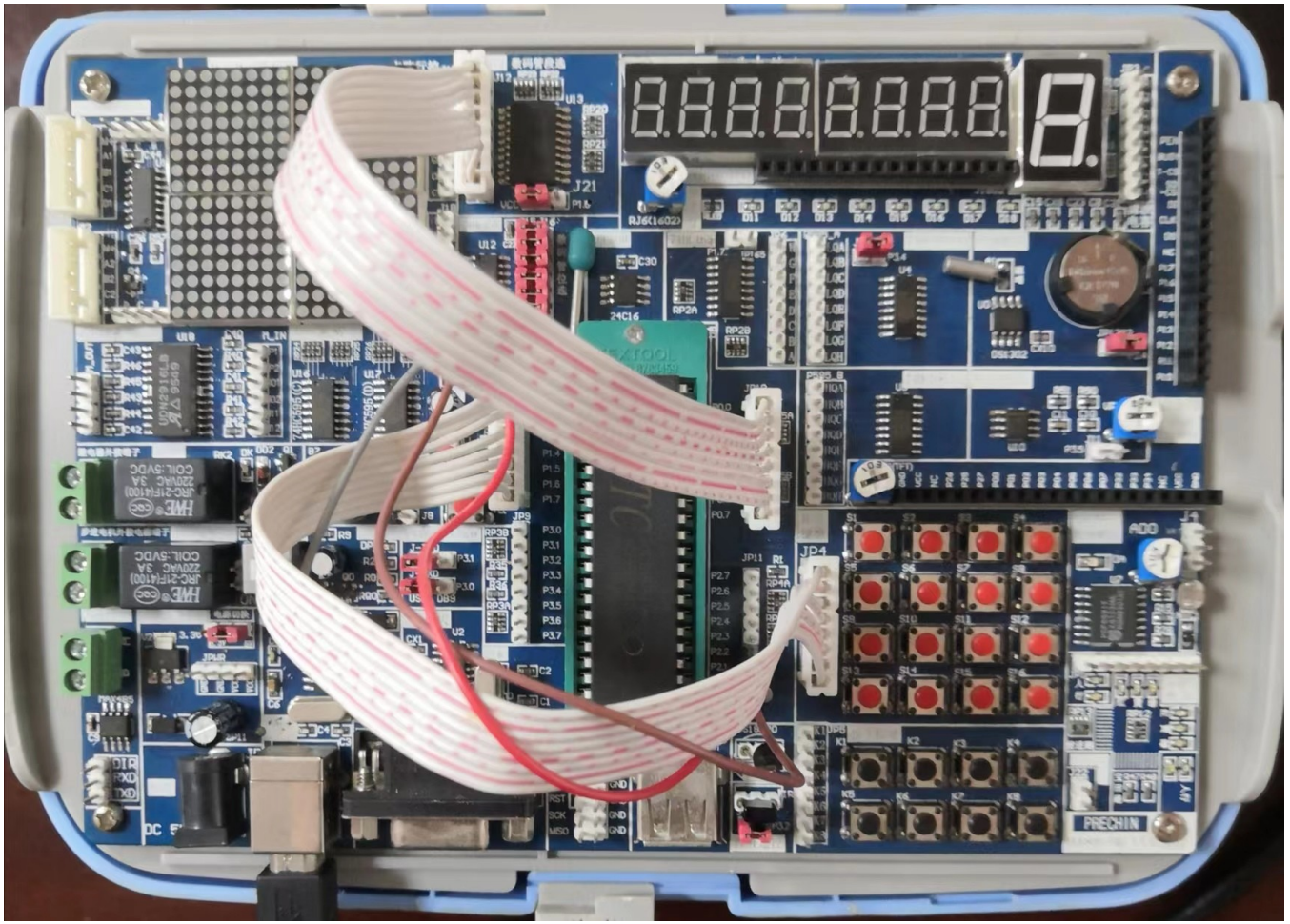
```
GPIO_KEY=0x0f;
if(GPIO_KEY!=0x0f)
{
    Delay10ms(1);
    a++;
    a=0;
    if(GPIO_KEY!=0x0f)
    {
        KeyState=1;//有按键按下
        //测试列
        GPIO_KEY=0X0F;
        Delay10ms(1);
        switch(GPIO_KEY)
        {
            case(0X07): KeyValue=0;break;
            case(0X0b): KeyValue=4;break;
            case(0X0d): KeyValue=8;break;
            case(0X0e): KeyValue=12;break;
            default:    KeyValue=17;    //检测出错回
        }
        //测试行
        GPIO_KEY=0XF0;
        Delay10ms(1);
        switch(GPIO_KEY)
        {
            case(0X70): KeyValue=KeyValue+3;break;
            case(0Xb0): KeyValue=KeyValue+2;break;
            case(0Xd0): KeyValue=KeyValue+1;break;
            case(0Xe0): KeyValue=KeyValue;break;
            default:    KeyValue=17;
        }
```

键盘显示实验一——动态移位显示键码

- C 语言代码见 QQ 群文件
- 理解电路原理，理解 C 语言程序

```
001
002 * 实 验 名          : 动态数码管按键移动显示试验
003 * 实验说明          :
004 * 连接方式          : 用排线将P0口和74H573旁边的J12连接起来
005 *                   * 将138译码器的A、B、C端分别接P2.0、P2.1、P2.2
006 *                   * 用排线将P1口和矩阵键盘旁边的JP4连接起来
007 * 注    意          :
008 ********************************************************************************/
009 #include<reg51.h>
010
011 //--定义使用的IO口--//
012 #define GPIO_DIG P0
013 #define GPIO_KEY P1
014
015 sbit LSA=P2^0;
016 sbit LSB=P2^1;
017 sbit LSC=P2^2;
```

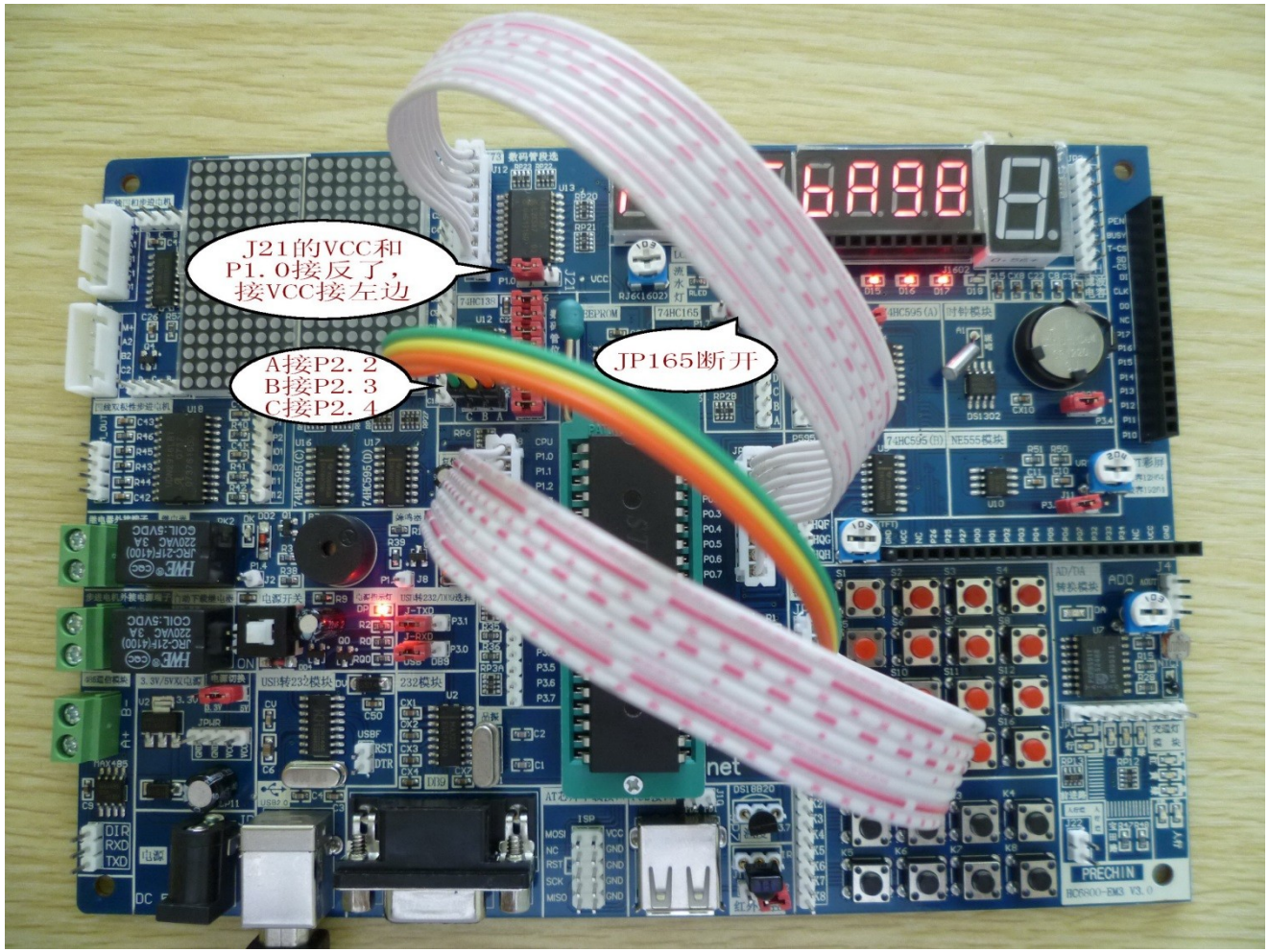
键盘显示实验一——动态移位显示键码



P2 端口低三位： 0->A； 1->B； 2->C

键盘显示实验二——汇编动态移位显示键码

环境与 C 语言移位实验一样，用汇编完整实现 C 语言程序功能



键盘显示实验二——汇编动态移位显示键码

```
//8位动态数码管显示移位键码汇编程序。P1接键盘，P2低三位三八译码后控制位（数码管），P0控制段显示

//循环显示，检查是否有按键，如果有，对段码缓存移位，并更新0存储单元的段码
            ORG 0000H
            AJMP MAIN      //上电后执行的第一条指令
            ORG  0030H
MAIN:       MOV R0,#00H    //R0,R1用于DISPLAY函数内部循环控制,R1指向段码表首地址
            MOV R1,#20H
            ACALL DISPLAY    //显示8个数码管，此时段码为上电初始值
            MOV R4, #0FFH    //当前键码,FF是不可能出现的键码，表示没有键按下
            ACALL KEYDOWN
            CJNE R4, #0FFH, CONTINUE //键码不等于FF，有键被按下
            AJMP MAIN
CONTINUE:   ACALL SHIFT    //调用子程序，把八个段码移一位
            MOV A, R4      //R4是键码
            MOV DPTR,#DIG_CODE
            MOVC A, @A+DPTR
            MOV 20H,A        //把当前按键键码对应段码写入20H存储单元
            AJMP MAIN

//20~27存储单元的内容移位子程序
SHIFT:                       //数码管对应数字顺移一位


            RET
//显示子程序，显示20~27存储单元的段码，存储单元上电初始值为0，故开始时数码管都不亮
DISPLAY:

            CJNE R0,#08H,DISPLAY    //循环完成8个数码管显示
            RET

KEYDOWN:                          //按键检查子程序，键码返回在R4中，如果R4为FF，表示没有键被按下
        补充键码检测程序

GOBACK:     RET
DIG_PLACE:  DB 00H,01H,02H,03H,04H,05H,06H,07H      //三八译码器的输入控制码表，对应输出为八个数码管的片选信号
DIG_CODE:   DB 3FH,06H,05bH,4fH,66H,6dH,7dH,07H,7fH,6fH,77H,7cH,39H,5eH,79H,71H     //共阴数码管段码表
            END
```